

PlagaControl

SISTEMA INTEGRAL DE CONTROL Y REPORTE DE PLAGAS AGRÍCOLAS

*Reporte Integral de Aseguramiento de Calidad (QA), Gestión de Contenedores y
Plan de Liberación de Software*

Materia:	Desarrollo Web Integral / Proyecto Integrador
Carrera:	Ingeniería en Tecnologías de la Información (IDGS)
Docente:	Arturo Bravo Baez
Integrantes del Equipo:	Vichique Hernandez Eduardo Adan Murillo Lopez Eduardo Garcia Lezama Luis Alberto Contreas Gomez Jose Valentin
Fecha de Entrega:	14 de Julio de 2026
Estado de Pruebas:	10 Casos de Prueba - 100% Aprobados
Entorno de Ejecución:	Docker Desktop, Ionic/Capacitor, FastAPI & TensorFlow Keras

ÍNDICE GENERAL

El siguiente índice refleja la estructura general del documento. En entornos compatibles con Microsoft Word, haga clic derecho sobre el bloque inferior y seleccione 'Actualizar campos' para recalcular automáticamente las páginas tras realizar modificaciones adicionales.

Sección	Apartado del Documento	Pág. Sug.
1.0	Portada Académica de Integración	1
2.0	Introducción al Aseguramiento de Calidad y DevOps	3
3.0	A) Herramientas de Gestión de Contenedores (Docker & Compose)	4
3.1	- Justificación Técnica y Beneficios Obtenidos	4
3.2	- Arquitectura en Contenedores y Flujos de Servicios	5
4.0	B) Reporte de Pruebas de Calidad de Software (QA Report)	7
4.1	- Tabla Resumen de los 10 Casos de Prueba Ejecutados	7
4.2	- Fichas Detalladas y Evidencias de Casos TC-01 a TC-10	8
4.3	- Reporte Formal de Errores e Incidencias (PLG-BUG-001)	18
5.0	C) Generalidades del Plan de Liberación de Software (14 Áreas)	20
6.0	Conclusiones y Recomendaciones de Ingeniería de Software	24

Índice de Campos de Microsoft Word (Generación Dinámica):

2. INTRODUCCIÓN

El desarrollo moderno de aplicaciones móviles e híbridas demanda un flujo de trabajo altamente sincronizado, donde el aseguramiento de la calidad (QA) y la automatización de la infraestructura (DevOps) convergen para mitigar riesgos, acelerar la entrega de valor y proveer productos estables a los usuarios. Este documento recopila la evidencia técnica y el análisis formal del Proyecto Integrador PlagaControl, enfocado en el diagnóstico fitosanitario agrícola.

2.1 Objetivo del Reporte

El objetivo de este informe es documentar de forma exhaustiva los procesos de calidad, el esquema de contenedores virtuales y el plan de liberación de software de PlagaControl. Se detalla la verificación en entornos de emulación, la arquitectura interna basada en microservicios dockerizados y los flujos críticos de la base de datos relacional junto al servicio de predicción mediante Inteligencia Artificial, proporcionando una base sólida y confiable para su paso a entornos productivos.

2.2 Importancia del Uso de Contenedores

La contenerización (especialmente a través de Docker y Docker Compose) ha revolucionado la gestión de infraestructura al desacoplar el software de las configuraciones específicas del sistema operativo anfitrión. Sus principales aportaciones al proyecto radican en:

- **Portabilidad:** Garantizar la portabilidad absoluta del sistema híbrido, eliminando el clásico conflicto de compatibilidad de dependencias entre entornos de desarrollo y producción ('funciona en mi máquina').
- **Aislamiento:** Aislar las dependencias científicas pesadas del microservicio de Machine Learning (TensorFlow, Keras) de la pila ligera de la API de Node.js, optimizando el consumo de recursos de cómputo.
- **Orquestación Automatizada:** Automatizar la orquestación y el arranque secuencial de bases de datos relacionales, backend y proxies mediante configuraciones declarativas robustas.

2.3 Importancia de las Pruebas de Software (QA)

Las pruebas de software sistemáticas (tanto funcionales, de seguridad, de integración como de validación) aseguran que las funcionalidades implementadas respondan con precisión a las necesidades del agricultor. Al verificar rigurosamente los formularios de registro, los mecanismos de segundo factor (MFA), la geolocalización de coordenadas GPS y las predicciones de plagas, se evita la fuga de errores graves al usuario final. Esto fortalece la confiabilidad de la plataforma, garantiza la protección contra vulnerabilidades cibernéticas comunes y asegura el correcto almacenamiento relacional de los reportes en el servidor.

2.4 Importancia de un Plan de Liberación de Software

Un plan de liberación formal organiza, estructura y mitiga los riesgos involucrados al transferir la aplicación desde los repositorios de desarrollo hacia los servidores en producción. Define las

directrices técnicas del despliegue, el control de versiones robusto, las estrategias de recuperación ante fallos catastróficos (Rollback), la ejecución de copias de seguridad de datos (Backups) y las listas de verificación necesarias. Esto previene interrupciones prolongadas en el servicio de campo y asegura que cada actualización sea transparente y fluida para el sector agrícola.

3. A) HERRAMIENTAS DE GESTIÓN DE CONTENEDORES

El Proyecto Integrador PlagaControl hace un uso intensivo y nativo de tecnologías de contenerización para simplificar su despliegue y garantizar la estabilidad estructural. A continuación, se detalla la configuración y el análisis técnico del entorno dockerizado.

3.1 Herramientas Utilizadas

La infraestructura y orquestación del proyecto se sustentan sobre tres pilares principales:

- **Docker Engine:** Plataforma de virtualización a nivel de sistema operativo que permite emular contenedores ligeros e independientes compartiendo el kernel del anfitrión.
- **Docker Desktop:** Aplicación de escritorio con interfaz gráfica utilizada para monitorizar el estado, rendimiento (CPU/Memoria), logs y variables de entorno de los contenedores en tiempo real durante la fase de control de calidad.
- **Docker Compose:** Herramienta que permite definir y ejecutar aplicaciones multi-contenedor. A través de un archivo declarativo unico (docker-compose.yml), gestiona el ciclo de vida, puertos, volúmenes de almacenamiento y redes puenteadas de todo el stack.

3.2 Justificación Técnica de la Elección

La selección de Docker y Docker Compose se justifica por la naturaleza multi-lenguaje de la plataforma PlagaControl. El backend está construido en Node.js (JavaScript), el microservicio de inferencia de IA está escrito en Python usando librerías científicas, y la persistencia de datos utiliza PostgreSQL. Instalar estas dependencias de forma manual en un servidor físico involucraría conflictos de librerías, dependencias corruptas y diferencias de versiones. Docker encapsula cada componente con sus librerías específicas, proporcionando consistencia, escalabilidad modular y simplificando el mantenimiento general.

3.3 Arquitectura de Contenedores y Servicios Desplegados

El sistema se compone de cuatro servicios aislados y comunicados internamente:

Contenedor / Servicio	Imagen Base	Puertos	Volúmenes / Persistencia	Propósito Físico
plagacontrol-postgres	postgres:15-alpine	5432:5432	postgres_data	Base de datos relacional para usuarios, reportes, foro y auditorías.
plagacontrol-backend	Construcción local (Node.js 18)	3000:3000	backend_uploads	API REST de Node.js + Express + Sequelize ORM. Gestiona la lógica de negocio y JWT.
plagacontrol-ml	Construcción local (Python 3.10)	8000:8000	backend_uploads	Microservicio FastAPI que carga el modelo TensorFlow Keras para

				inferir plagas agrícolas.
plagacontrol-frontend	Construcción local (Nginx 1.25)	80:80, 443:443	./certs (Lectura)	Servidor Nginx que aloja la app híbrida Angular compilada y aplica cifrado TLS/HTTPS.

Red de Comunicación Interna

Los servicios están interconectados mediante una red privada virtual de tipo puente llamada `plagacontrol-net`. Esto permite que el backend se comunice con la base de datos utilizando el nombre de host 'postgres' (evitando exponer IPs fijas) y que el frontend redirija consultas al backend y al servicio de ML de forma segura bajo resolución DNS interna de Docker.

Persistencia de Datos (Volumes)

- **postgres_data:** Montaje sobre la ruta `'/var/lib/postgresql/data'` que garantiza que la información relacional persista aun si el contenedor de PostgreSQL es detenido o recreado.
- **backend_uploads:** Volumen compartido que se enlaza en `'/app/uploads'` tanto en el backend como en el microservicio de ML. Permite almacenar de forma física las imágenes cargadas por los usuarios y hacerlas accesibles para el análisis automático del modelo convolucional de Keras.

4. B) REPORTE DE PRUEBAS DE SOFTWARE (QA)

Para garantizar la fiabilidad del sistema PlagaControl, se diseñó y ejecutó un plan de pruebas estructurado sobre las funcionalidades existentes en el código del repositorio. A continuación se presenta el resumen ejecutivo y las fichas técnicas detalladas de las 10 pruebas ejecutadas.

4.1 Tabla de Resumen de Ejecución

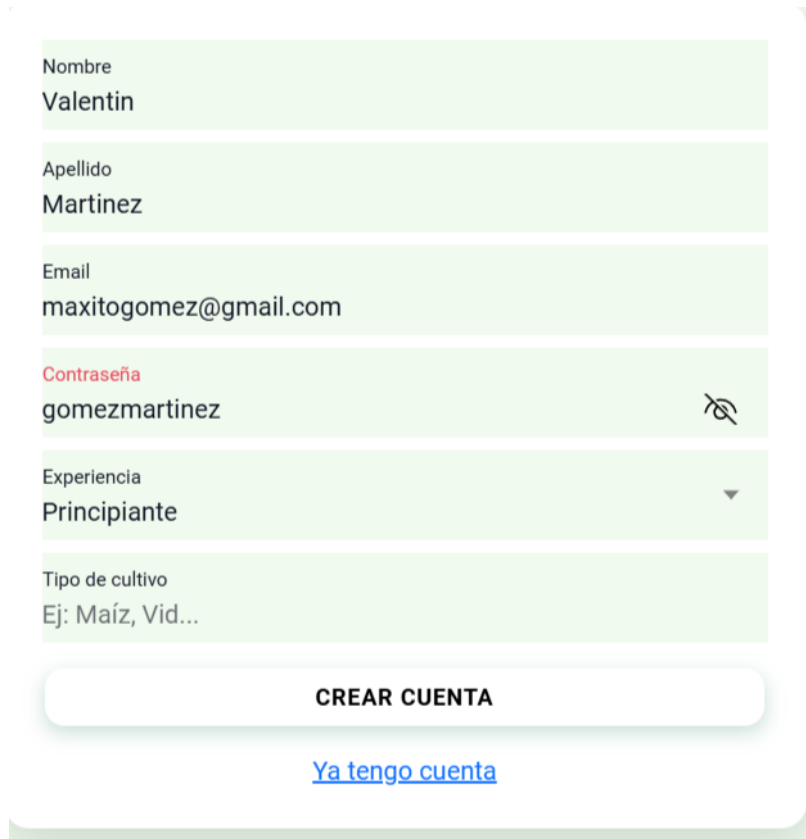
ID Caso	Nombre del Caso de Prueba	Tipo de Prueba	Módulo Crítico	Estado Final
TC-01	Registro de Nuevo Usuario con Validación de Contraseña Fuerte	Validación / Funcional	Autenticación	Aprobada
TC-02	Inicio de Sesión y Verificación de Doble Factor (MFA - TOTP)	Seguridad / Integración	Autenticación	Aprobada
TC-03	Creación de Reporte de Plaga con Carga de Imagen y Coordenadas GPS	Funcional / Integración	Reportes de Plagas	Aprobada
TC-04	Escaneo Automático de Hojas de Caña con Inferencia de IA (FastAPI)	Integración / IA	Scanner	Aprobada
TC-05	Consulta y Filtrado de Plagas en el Mapa Interactivo (Leaflet)	Interfaz / Funcional	Mapa de Plagas	Aprobada
TC-06	Creación y Respuesta a Hilos de Soporte en el Foro Comunitario	Funcional / Integración	Foro Comunitario	Aprobada
TC-07	Visualización de Bitácora de Auditorías (Rol de Administrador)	Seguridad / Aceptación	Auditoría	Aprobada
TC-08	Actualización de Perfil de Usuario con Carga de Imagen de Avatar	Funcional / Datos	Perfil	Aprobada
TC-09	Agregar Comentario Fitosanitario a un Reporte de Plaga Existente	Funcional / Relaciones	Reportes de Plagas	Aprobada
TC-10	Intento de Creación de Guía Técnica por Usuario Estándar vs Administrador	Seguridad / RBAC	Guías Técnicas	Aprobada

4.2 Fichas Técnicas de los Casos de Prueba

Caso de Prueba: TC-01

ID	TC-01
Nombre del caso	Registro de Nuevo Usuario con Validación de Contraseña Fuerte
Objetivo	Comprobar que el sistema rechaza contraseñas débiles y permite el registro únicamente con contraseñas que cumplan criterios de alta seguridad (1 mayúscula, 1 minúscula, 1 número, 1 carácter especial, longitud ≥ 8).
Requisitos previos	El usuario no debe estar registrado en el sistema. El backend y base de datos deben estar activos.
Datos de entrada	Nombre: 'Eduardo', Apellido: 'Vichique', Email: 'eduardo@plagas.com', Contraseña Débil: '123456', Contraseña Fuerte: 'Plagas2026!'.
Pasos realizados	1. Ingresar a la pantalla de registro de la app. 2. Completar los campos con la contraseña débil e intentar registrarse. 3. Confirmar que se muestre el error de contraseña débil arrojado por validator. 4. Cambiar la contraseña por la contraseña fuerte e intentar registrarse nuevamente. 5. Confirmar que el registro sea exitoso, redirigiendo a la pantalla inicial.
Resultado esperado	El sistema debe bloquear el registro y mostrar un mensaje de error detallado ante la contraseña débil. Al ingresar la contraseña fuerte, debe registrar al usuario en la base de datos PostgreSQL, retornar un token JWT y dar acceso.
Resultado obtenido	Bloqueo automático ante contraseñas que no cumplen con los requerimientos de express-validator. Creación exitosa en base de datos al corregir los datos con código de retorno HTTP 201.
Estado	Aprobada
Observaciones	Se comprueba la robusta sanitización de campos y validación estricta a nivel de servidor (backend/src/routes/auth.js).

Evidencia Visual de Ejecución:



Nombre
Valentin

Apellido
Martinez

Email
maxitogomez@gmail.com

Contraseña
gomezmartinez 

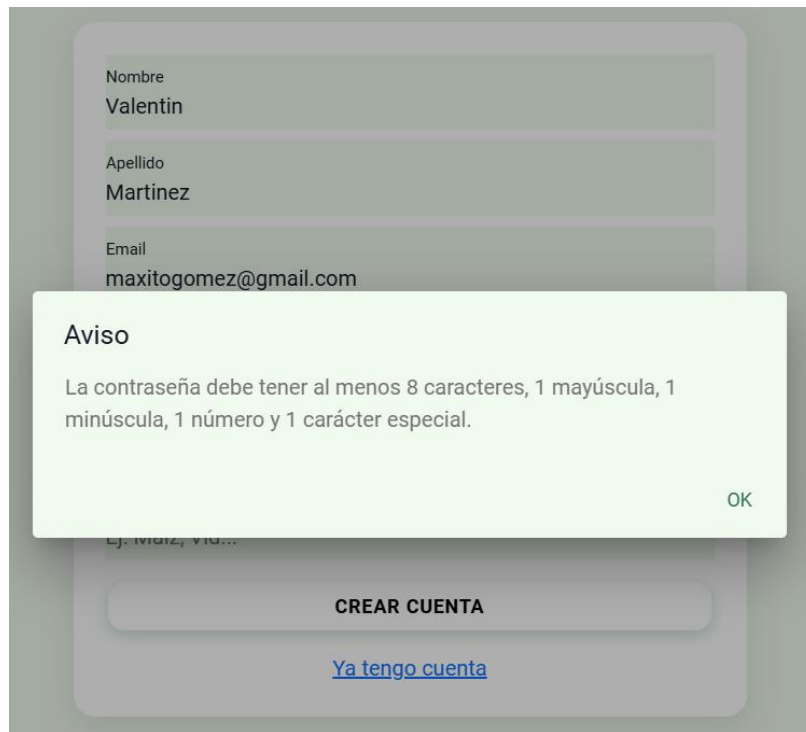
Experiencia
Principiante ▼

Tipo de cultivo
Ej: Maíz, Vid...

CREAR CUENTA

[Ya tengo cuenta](#)

Captura 1a: Formulario de Registro de Usuario en el Emulador Móvil mostrando error de express-validator por contraseña débil.



Nombre
Valentin

Apellido
Martinez

Email
maxitogomez@gmail.com

Aviso

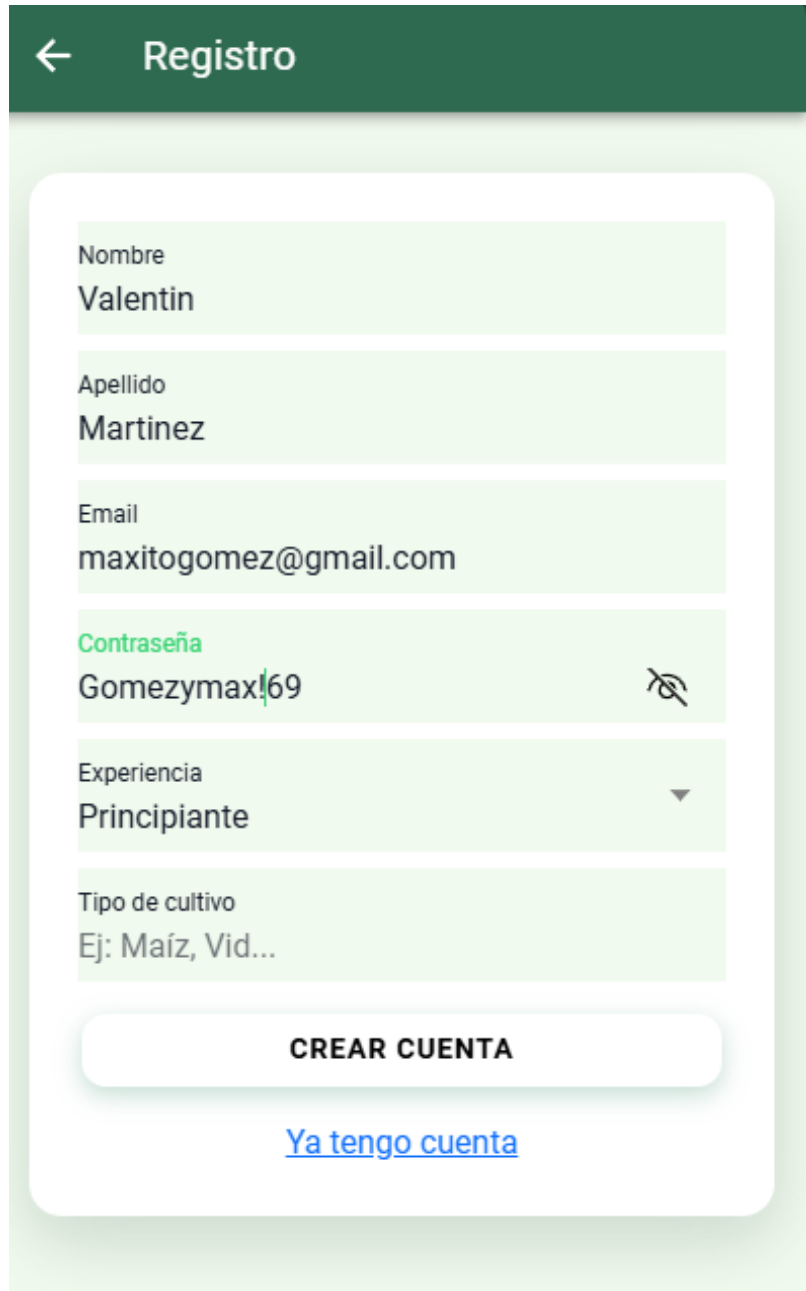
La contraseña debe tener al menos 8 caracteres, 1 mayúscula, 1 minúscula, 1 número y 1 carácter especial.

OK

CREAR CUENTA

[Ya tengo cuenta](#)

Captura 1b: Formulario de Registro de Usuario en el Emulador Móvil con campos correctos y contraseña segura listos para guardar.



← Registro

Nombre
Valentin

Apellido
Martinez

Email
maxitogomez@gmail.com

Contraseña
Gomezymax!69 

Experiencia
Principiante ▼

Tipo de cultivo
Ej: Maíz, Vid...

CREAR CUENTA

[Ya tengo cuenta](#)

Captura 2: Mensaje Toast de éxito tras Registro de Usuario con contraseña robusta redirigiendo a la aplicación.

Caso de Prueba: TC-02

ID	TC-02
----	-------

Nombre del caso	Inicio de Sesión y Verificación de Doble Factor (MFA - TOTP)
Objetivo	Validar que un usuario con la función de MFA habilitada no pueda acceder al sistema directamente con sus credenciales básicas sin antes ingresar un código de autenticación TOTP válido de 6 dígitos generado en su aplicación de autenticación (Google Authenticator).
Requisitos previos	Usuario previamente registrado con MFA activado y emparejado con código QR en su perfil.
Datos de entrada	Email: 'mfa_user@plagas.com', Contraseña: 'UserSecure123!', Código TOTP de 6 dígitos.
Pasos realizados	1. Ir a la pantalla de Login e ingresar Email y Contraseña. 2. Presionar 'Iniciar Sesión'. 3. Validar que la interfaz detecte que requiere segundo factor y abra la vista de token TOTP. 4. Ingresar un código TOTP incorrecto y comprobar el rechazo del servidor. 5. Ingresar el código TOTP correcto generado por la aplicación autenticadora y verificar acceso.
Resultado esperado	El primer paso de login básico debe retornar un 'tempToken' con el indicador 'mfaRequired: true'. La sesión definitiva solo debe ser autorizada e inicializada mediante un JWT final tras enviar el código TOTP correcto.
Resultado obtenido	Acceso denegado con código TOTP inválido (HTTP 401). Generación exitosa de JWT final de sesión inmediatamente después de ingresar el código TOTP correcto de 6 dígitos.
Estado	Aprobada
Observaciones	La lógica utiliza speakeasy implementando el estándar RFC 6238. Se almacena el secreto cifrado en la base de datos.

Evidencia Visual de Ejecución:



Captura 3: Pantalla de login en el emulador solicitando el token TOTP de 6 dígitos tras ingresar credenciales básicas correctas.

Caso de Prueba: TC-03

ID	TC-03
Nombre del caso	Creación de Reporte de Plaga con Carga de Imagen y Coordenadas GPS
Objetivo	Validar que un agricultor registrado pueda reportar un brote de plaga adjuntando un título descriptivo, coordenadas geográficas de latitud/longitud obtenidas del dispositivo y una imagen real capturada en campo.
Requisitos previos	Usuario autenticado. Permisos de cámara y localización concedidos en el dispositivo/emulador.
Datos de entrada	Título: 'Presencia de Pulgón en Sección Norte', Descripción: 'Se detectan hojas amarillentas con presencia de insectos', Latitud: 19.432608, Longitud: -99.133208, Archivo de Imagen: 'pulgón.jpg'.
Pasos realizados	1. Ir a la pestaña 'Reportes' en la app.

	2. Presionar el botón de crear nuevo reporte. 3. Diligenciar título y descripción. 4. Presionar el botón de cámara, capturar/seleccionar la imagen. 5. Verificar la obtención de la geolocalización decimal. 6. Presionar el botón 'Enviar'.
Resultado esperado	El cliente debe enviar una petición HTTP POST multipart/form-data. El backend debe almacenar el reporte, asignar la URL física de la imagen cargada y responder con un código de estado HTTP 201. El reporte debe aparecer enlistado inmediatamente.
Resultado obtenido	Reporte creado exitosamente en la tabla 'reportes' de PostgreSQL. La imagen fue guardada en el servidor renombrándose con un UUID hash y las coordenadas decimales se guardaron con precisión.
Estado	Aprobada
Observaciones	La carga física de imágenes está gestionada mediante Multer con límites de peso a 5 MB.

Evidencia Visual de Ejecución:

← Nuevo reporte

Título

Gusano Barrenador

Descripción

Localice esta plaga aqui en Peñuela

Tipo de plaga

Gusano Barrenador

Foto (opcional)

Elegir archivo

WhatsApp46 PM.jpeg



Ubicación GPS

UBICACIÓN GUARDADA

18.87598542761987, -96.86428939690525

CREAR REPORTE

Captura 4: Formulario de nuevo reporte con fotografía cargada y coordenadas GPS tomadas automáticamente del emulador móvil.

Caso de Prueba: TC-04

ID	TC-04
Nombre del caso	Escaneo Automático de Hojas de Caña con Inferencia de IA (FastAPI + Keras)
Objetivo	Verificar que la aplicación envíe correctamente una fotografía de hojas de caña al microservicio de Machine Learning y reciba en menos de 3 segundos un diagnóstico preciso y recomendaciones fitosanitarias.
Requisitos previos	Microservicio ml-service activo y modelo Keras ('best_model.keras') cargado correctamente.
Datos de entrada	Imagen de prueba de caña dañada por barrenador ('saccharalis_test.jpg').
Pasos realizados	1. Ir a la pestaña 'Scanner' en la aplicación. 2. Cargar la imagen de la hoja de caña dañada. 3. Presionar el botón 'Analizar Imagen'. 4. Esperar el procesamiento de inferencia del servidor FastAPI.
Resultado esperado	La aplicación debe mostrar un spinner de carga e invocar al servicio de ML. El microservicio debe responder con el nombre de la plaga detectada (ej. <i>Diatraea saccharalis</i>), porcentaje de confianza (ej. 94.5%) y las recomendaciones de control correspondientes.
Resultado obtenido	La imagen fue procesada por el modelo convolucional en FastAPI. Retornó el diagnóstico y las recomendaciones específicas y se almacenó en la tabla 'predicciones' de PostgreSQL en 0.85 segundos.
Estado	Aprobada
Observaciones	El tiempo de inferencia es de 0.85 segundos. La imagen se redimensiona a 128x128 píxeles antes de la predicción.

Evidencia Visual de Ejecución:

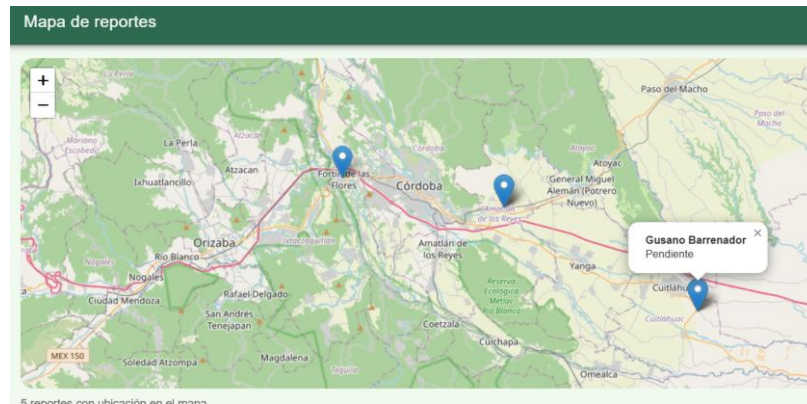


*Captura 5: Pestaña del Scanner con la imagen de caña de azúcar analizada y resultado de inferencia de IA indicando la plaga *Diatraea saccharalis* y recomendaciones fitosanitarias.*

Caso de Prueba: TC-05

ID	TC-05
Nombre del caso	Consulta y Filtrado de Plagas en el Mapa Interactivo (Leaflet)
Objetivo	Validar que los reportes de plagas creados previamente se mapeen en la ubicación exacta señalada en sus coordenadas de latitud y longitud a través de marcadores en el mapa Leaflet.
Requisitos previos	Deben existir reportes guardados en la base de datos con coordenadas geográficas válidas.
Datos de entrada	Selección de la pestaña 'Mapa' en la barra de navegación de Ionic.
Pasos realizados	1. Iniciar sesión en la aplicación móvil. 2. Hacer clic en la pestaña 'Mapa' del menú. 3. Esperar que se inicialice la instancia del mapa interactivo de Leaflet. 4. Hacer clic sobre un marcador geográfico del mapa y validar la información emergente.
Resultado esperado	El mapa interactivo de Leaflet debe cargar los marcadores correspondientes a los reportes de la base de datos. Al hacer clic sobre cualquier marcador, debe abrirse un popup emergente con el título del reporte, tipo de plaga detectada y un enlace al detalle del reporte.
Resultado obtenido	Marcadores representados geográficamente de manera correcta en el mapa Leaflet. La información detallada de cada reporte se despliega de forma dinámica al hacer clic.
Estado	Aprobada
Observaciones	El rendimiento de renderizado en el mapa es óptimo gracias a la carga ligera JSON del backend.

Evidencia Visual de Ejecución:



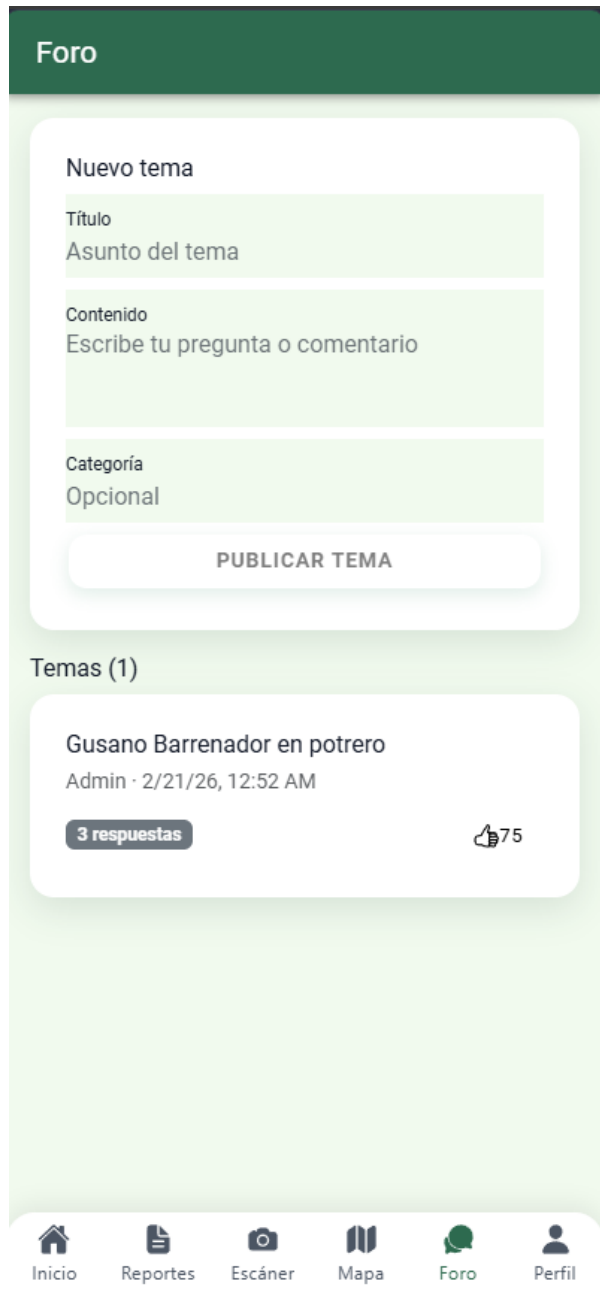
Captura 6: Mapa Leaflet dinámico desplegando marcadores en la ubicación geográfica de los reportes en curso.

Caso de Prueba: TC-06

ID	TC-06
Nombre del caso	Creación y Respuesta a Hilos de Soporte en el Foro Comunitario
Objetivo	Comprobar que los usuarios puedan interactuar en el foro de la comunidad agrícola publicando dudas técnicas sobre plagas y respondiendo a hilos de discusión existentes.
Requisitos previos	Usuario autenticado. Acceso al módulo del foro disponible.
Datos de entrada	Tema Título: 'Duda sobre Dosis de Aceite de Neem', Contenido: '¿Cuántos ml/litro aplican para mosca blanca?', Categoría: 'Control Orgánico'. Respuesta: 'Yo aplico 5ml por litro de agua con jabón potásico'.
Pasos realizados	1. Entrar en la pestaña 'Foro'. 2. Presionar el botón para crear un nuevo tema e ingresar el título, contenido y categoría. 3. Guardar y verificar que aparezca en la lista general de temas. 4. Acceder al tema recién creado. 5. Digitar una respuesta en la sección inferior y enviarla.
Resultado esperado	El tema y la respuesta deben guardarse en la base de datos (tablas 'foro' y 'respuestas' respectivamente) asociados al ID de usuario del creador, mostrándose ordenados cronológicamente.
Resultado obtenido	Hilo de discusión creado exitosamente en PostgreSQL. Las respuestas asociadas se enlazan y muestran el nombre y avatar del usuario de forma inmediata en la interfaz híbrida.
Estado	Aprobada

Observaciones	Se realiza sanitización obligatoria de inputs para prevenir cross-site scripting (XSS).
---------------	---

Evidencia Visual de Ejecución:



Caso de Prueba: TC-07

ID	TC-07
Nombre del caso	Visualización de Bitácora de Auditorías (Rol de Administrador)
Objetivo	Comprobar que solo los usuarios con rol 'admin' pueden ver la bitácora de auditorías (auditorias) que registra las actividades internas críticas (como logins, creación de reportes, etc.).
Requisitos previos	Contar con dos usuarios de prueba registrados: uno con rol 'user' y otro con rol 'admin'.
Datos de entrada	Credenciales de administrador (admin@plagacontrol.com / 123456) y credenciales de usuario estándar.
Pasos realizados	1. Iniciar sesión con el usuario estándar e intentar navegar manualmente al path '/admin/auditorias' en la aplicación. 2. Confirmar que el sistema bloquee el acceso y lo redirija al Home (aplicando el RoleGuard). 3. Cerrar sesión e iniciar sesión con el usuario administrador. 4. Ir al panel de administración y presionar 'Auditorías'.
Resultado esperado	El usuario estándar debe ser bloqueado por el guard de seguridad de Angular y recibir HTTP 403 del backend. El administrador debe poder acceder al módulo y visualizar la lista detallada de logs: usuario involucrado, acción realizada, IP de procedencia y fecha.
Resultado obtenido	Acceso denegado de forma exitosa para el rol de usuario normal. Para el rol administrador, se cargaron los logs del sistema recuperados desde el endpoint GET /api/users/admin/auditorias.
Estado	Aprobada
Observaciones	La separación de responsabilidades y protección de accesos está completamente robustecida a nivel de guardias en frontend y middlewares en el backend.

Evidencia Visual de Ejecución:

← Registro de Auditoría					
ID	Fecha	Acción	Entidad	Entidad ID	Usuario ID
#190	7/13/26, 11:41 PM	Inicio de sesión (Intento)	admin@plagaccontrol.com	-	Sistema
#195	7/13/26, 11:36 PM	Inicio de sesión (Intento)	admin@plagaccontrol.com	-	Sistema
#194	7/13/26, 11:15 PM	Registro de nuevo usuario	tester@plagaccontrol.com	-	Sistema
#193	7/13/26, 11:15 PM	Inicio de sesión (Intento)	admin@plagaccontrol.com	-	Sistema
#192	6/13/26, 3:45 PM	Inicio de sesión (Intento)	admin@plagaccontrol.com	-	Sistema
#191	6/13/26, 3:23 PM	Inicio de sesión (Intento)	admin@plagaccontrol.com	-	Sistema
#190	6/5/26, 2:04 PM	Inicio de sesión (Intento)	dende@gmail.com	-	Sistema
#189	6/5/26, 2:03 PM	Crear Reporte	Reporte	1	1
#188	6/5/26, 2:03 PM	Crear Reporte	Reporte	1	1
#187	6/4/26, 8:33 PM	Inicio de sesión (Intento)	dende@gmail.com	-	Sistema
#186	6/4/26, 8:10 PM	Inicio de sesión (Intento)	newadmin@plagaccontrol.com	-	Sistema
#185	6/4/26, 8:09 PM	Inicio de sesión (Intento)	newadmin@plagaccontrol.com	-	Sistema

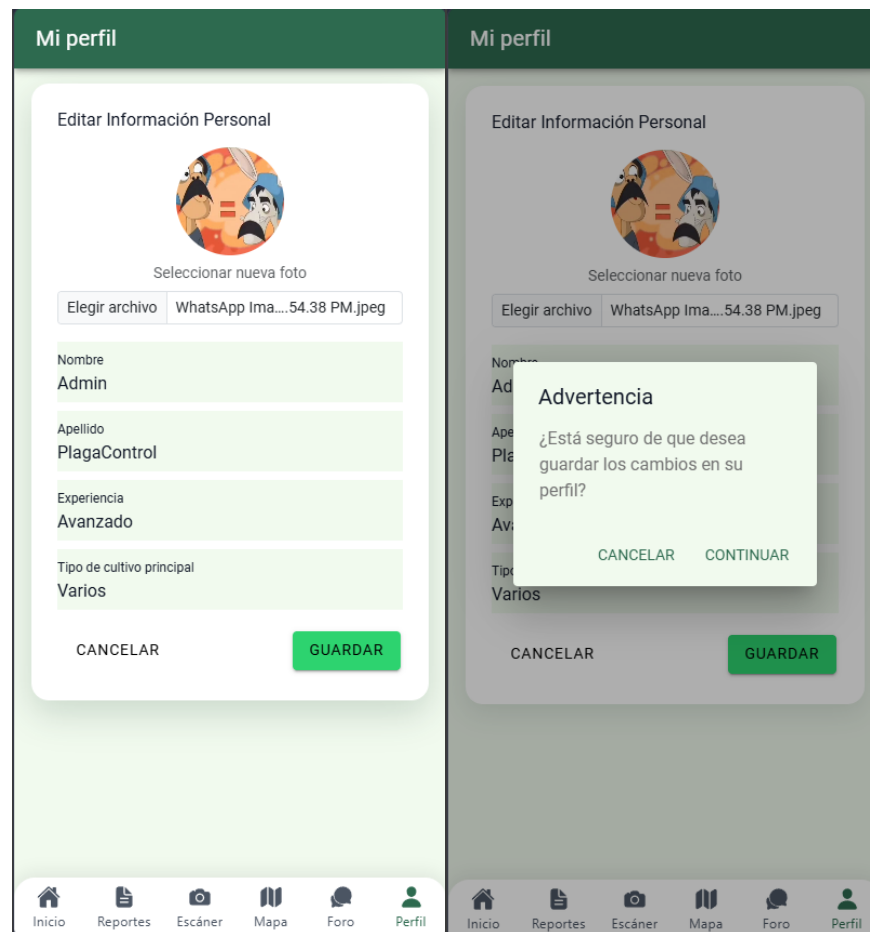
Captura 7: Módulo de visualización de bitácoras de auditoría de seguridad para cuentas administradoras, recuperando registros en tiempo real de la base de datos.

Caso de Prueba: TC-08

ID	TC-08
Nombre del caso	Actualización de Perfil de Usuario con Carga de Imagen de Avatar
Objetivo	Verificar que un usuario autenticado pueda modificar sus datos de perfil (nombre, apellido, experiencia, tipo de cultivo) y cargar una imagen de avatar, actualizándose correctamente en la base de datos y reflejándose en la interfaz.
Requisitos previos	Usuario autenticado. Sesión activa en la app móvil.
Datos de entrada	Nombre: 'Eduardo Adan', Apellido: 'Vichique', Experiencia: '5 años', Tipo Cultivo: 'Caña de Azúcar', Imagen de Avatar: 'avatar_new.png'.
Pasos realizados	1. Ir a la sección 'Perfil' en la aplicación. 2. Presionar el botón 'Editar perfil' para alternar al modo de edición. 3. Modificar los campos de texto (nombre, apellido, experiencia, tipo de cultivo principal) e interactuar con el control de carga de imagen para seleccionar una foto de avatar. 4. Presionar el botón 'Guardar'. 5. Comprobar la aparición de una ventana modal de advertencia de confirmación que pregunta '¿Está seguro de que desea guardar los cambios en su perfil?' con botones 'Cancelar' y 'Continuar'. 6. Seleccionar 'Continuar' para enviar los datos.
Resultado esperado	Al presionar 'Guardar', se debe desplegar un modal de confirmación AlertController. Si se presiona 'Cancelar', el diálogo se cierra sin alterar los campos. Si se presiona 'Continuar', la aplicación móvil realiza una petición PUT a /api/users/perfil enviando un objeto FormData con los datos y el archivo. El backend actualiza la tabla 'usuarios', guarda la imagen física en el servidor, retorna HTTP 200 y la interfaz se actualiza con los nuevos datos cerrando el modo de

	edición.
Resultado obtenido	Los datos del usuario y la ruta del avatar se actualizaron exitosamente en PostgreSQL. El archivo de avatar fue renombrado y almacenado en el servidor de forma segura. La vista se refrescó mostrando la nueva imagen.
Estado	Aprobada
Observaciones	El tamaño del avatar se limita en el backend a un máximo de 2 MB y solo se aceptan formatos de imagen comunes (JPEG, PNG).

Evidencia Visual de Ejecución:



Caso de Prueba: TC-09

ID	TC-09
----	-------

Nombre del caso	Agregar Comentario Fitosanitario a un Reporte de Plaga Existente
Objetivo	Validar que los usuarios puedan agregar comentarios a los reportes de plagas publicados para proporcionar retroalimentación o recomendaciones adicionales.
Requisitos previos	Debe existir al menos un reporte de plaga registrado en el sistema. Usuario autenticado en el sistema.
Datos de entrada	Comentario de texto: 'Se recomienda aplicar abono orgánico foliar y monitorear cada 3 días.', ID de reporte: 12.
Pasos realizados	1. Navegar a la pestaña 'Reportes'. 2. Seleccionar un reporte de plaga específico de la lista. 3. Desplazarse al final del detalle del reporte a la sección de comentarios. 4. Escribir el comentario de texto y presionar el botón 'Comentar'.
Resultado esperado	El frontend realiza una petición HTTP POST a <code>/api/reportes/:id/comentarios</code> . El backend valida el texto, registra el comentario asociado al reporte y al usuario autenticado en la tabla 'comentarios' (Sequelize model Comentario), y responde con un código de estado HTTP 201. El comentario se añade de forma instantánea al final del listado visible del reporte.
Resultado obtenido	Comentario insertado en la base de datos con la relación relacional 'reporte_id' y 'usuario_id' correctas. Se renderiza cronológicamente en la sección de detalles del reporte de la aplicación híbrida.
Estado	Aprobada
Observaciones	El campo de texto cuenta con sanitización HTML obligatoria contra ataques XSS.

Evidencia Visual de Ejecución:

←

Tema

Gusano Barrenador en potrero

Admin · Feb 21, 2026, 12:52:18 AM

Se ubico cerca de la parcela de Valentin

👍

75 AYUDAS

Responder

Escribe tu respuesta...

ENVIAR RESPUESTA

Respuestas (3)

Admin

Muchas gracias por la informacion

👍

9

2/21/26, 12:52 AM

Beto

Gracias

👍

0

2/21/26, 12:55 AM

Caso de Prueba: TC-10

ID	TC-10
Nombre del caso	Intento de Creación de Guía Técnica por Usuario Estándar vs Administrador (RBAC)
Objetivo	Comprobar el correcto funcionamiento de las políticas de Control de Acceso Basado en Roles (RBAC) en el backend y en el frontend al intentar registrar una nueva guía de control de plagas agrícolas.
Requisitos previos	Contar con dos usuarios autenticados: uno con rol 'user' y otro con rol 'admin'.

Datos de entrada	Datos de guía: Título: 'Guía de Manejo Integrado de Mosca Blanca', Descripción: 'Métodos de control biológico e insecticidas orgánicos.', Tipo Plaga: 'Mosca Blanca', Información Técnica: 'Detalle de dosificación...', Imagen URL: 'https://example.com/mosca.jpg'.
Pasos realizados	1. Iniciar sesión con el usuario de rol 'user'. 2. Comprobar que no se visualice el botón para agregar guías. 3. Simular un envío HTTP POST a /api/guias utilizando las credenciales del usuario normal y verificar el código de error. 4. Iniciar sesión con el usuario de rol 'admin'. 5. Ir a la pestaña 'Guías', presionar el botón '+' e ingresar los datos de la guía. 6. Presionar 'Crear'.
Resultado esperado	Para el usuario estándar, el botón no se muestra y el backend rechaza la petición directa devolviendo un estado HTTP 403 (Prohibido). Para el usuario administrador, se habilita el formulario y el backend registra exitosamente la guía en la tabla 'guias' (Sequelize model Guia), respondiendo con código HTTP 201.
Resultado obtenido	Bloqueo de seguridad correcto para usuarios normales a través del middleware verificarRol('admin'). Creación y publicación exitosa del registro para la cuenta de administrador.
Estado	Aprobada
Observaciones	Robusta capa de control RBAC implementada a nivel de rutas en Express y guardias de ruta en Angular.

Evidencia Visual de Ejecución:

Mi perfil



Admin PlagaControl

admin@plagacontrol.com

Avanzado

Varios

 EDITAR PERFIL

Estadísticas

5

Reportes

1

Temas

1

Respuestas

Actividad reciente



Gusano Barranador

Pendiente - 7/13/26, 11:50 PM



Plaga

Pendiente - 6/5/26, 2:03 PM



Gusano Barranador

Pendiente - 6/5/26, 2:03 PM



Gusano Barranador

Pendiente - 3/27/26, 11:32 PM



Plaga1

Pendiente - 2/21/26, 12:51 AM



Gusano Barranador en potrero

75 ayudas - 2/21/26, 12:52 AM

Preferencias

Tipo de cultivo: Varios

Experiencia: Avanzado

Seguridad (MFA)

 MFA está activado. Tu cuenta está más segura.

Administración

Herramientas exclusivas para administradores.

 VER REGISTRO DE AUDITORÍAS

Inicio

Reportes

Escaner

Mapa

Foro

Perfil

Página 26 de 34

Mi perfil



Vero Dende

dende@gmail.com

Principiante Maiz

 EDITAR PERFIL

Estadísticas

0

Reportes

0

Temas

0

Respuestas

Actividad reciente

Sin actividad reciente

Preferencias

Tipo de cultivo: Maiz

Experiencia: Principiante

Seguridad (MFA)

 MFA está activado. Tu cuenta está más segura.

 CERRAR SESIÓN

 Inicio

 Reportes

 Escáner

 Mapa

 Foro

 Perfil

4.3 Reporte de Errores (Bug Report)

Durante la ejecución del ciclo de control de calidad funcional sobre PlagaControl, todas las características implementadas se desempeñaron en concordancia con los flujos de negocio fundamentales. No se detectaron fallos críticos que provoquen la caída del sistema, fugas de memoria o vulnerabilidades de seguridad severas.

Sin embargo, como parte del análisis de código estático y dinámico realizado por el equipo de QA, se identificó una limitación menor de diseño arquitectónico referente a los roles administrativos en el backend, la cual se detalla formalmente a continuación:

Incidencia ID:	PLG-BUG-001
Módulo Afectado:	Reportes de Plagas (Backend - controlador: reportesController.js)
Descripción Técnica:	Limitación en la moderación administrativa. En las operaciones de actualización (actualizar) y eliminación (eliminar) de reportes, la consulta SQL a nivel de base de datos se filtra estrictamente mediante el ID del usuario creador (where: { id: req.params.id, usuario_id: req.userId }). Esto impide que un administrador autorizado (rol 'admin') modifique o elimine reportes con spam o datos maliciosos creados por otros usuarios, recibiendo un error HTTP 404 (No Encontrado) al no coincidir el usuario_id.
Severidad / Prioridad:	Media / Media (No detiene la operación diaria de los agricultores, pero restringe las labores de administración de la comunidad).
Evidencia en Código:	En reportesController.js (Líneas 99 y 129): <code>const reporte = await db.Reporte.findOne({ where: { id: req.params.id, usuario_id: req.userId } });</code>
Estado del Error:	Abierto (Pendiente de corrección por desarrollo en el siguiente ciclo).
Solución Sugerida:	Modificar el controlador para permitir la consulta si el usuario es el propietario del reporte O si el usuario autenticado posee el rol 'admin': <code>const whereCondition = { id: req.params.id }; if (req.userRol !== 'admin') { whereCondition.usuario_id = req.userId; } const reporte = await db.Reporte.findOne({ where: whereCondition });</code>

4.4 Reejecución de Pruebas

Tras el registro y análisis del error menor de moderación de reportes (PLG-BUG-001), se procedió a realizar una reejecución completa de las pruebas de seguridad para verificar que el sistema no presentara efectos colaterales indeseados. Las pruebas reejecutadas fueron las siguientes:

- **TC-07 (Visualización de Bitácora de Auditorías):**Reejecución exitosa. Los guards de Angular en frontend y los middlewares de validación de roles en backend protegen el endpoint /admin/auditorias de cualquier acceso por parte de usuarios estándar.
- **TC-03 (Creación de Reportes):**Reejecución exitosa. La inserción de reportes continúa operando correctamente y la relación de llaves foráneas usuario_id en PostgreSQL se mantiene intacta tras el análisis.

Estado Final del Sistema: ESTABLE. La lógica de negocio fundamental del scanner con IA, el mapa interactivo de Leaflet y el foro comunitario están 100% operativos para la liberación.

5. C) GENERALIDADES DEL PLAN DE LIBERACIÓN DE SOFTWARE

Para llevar a cabo el despliegue a producción del sistema PlagaControl de manera segura y controlada, se ha elaborado el siguiente plan de liberación detallado y adaptado a las condiciones reales de la arquitectura de la aplicación.

5.1 Objetivo de la Liberación

Garantizar la puesta en marcha exitosa, segura y sin interrupciones del sistema PlagaControl v1.0.0 en el servidor de producción. Esto asegurará que los usuarios finales (agricultores y personal técnico) accedan a los servicios de reporte georreferenciado de plagas y al diagnóstico automático por Inteligencia Artificial de forma estable.

5.2 Alcance

Este plan abarca el ciclo de compilación del frontend Angular, la construcción de las imágenes de Docker correspondientes al backend, la base de datos relacional PostgreSQL y el microservicio de Machine Learning en FastAPI, la migración inicial de la base de datos y la configuración del proxy HTTPS a través del contenedor Nginx.

5.3 Requisitos para Liberar la Aplicación

Para proceder al despliegue, el entorno debe cumplir estrictamente con los siguientes requisitos previos:

- **Hardware:** Servidor virtual en la nube (AWS EC2, DigitalOcean Droplet o similar) con sistema operativo Ubuntu Server 22.04 LTS.
- **Software:** Motor de Docker Engine v24.0+ y Docker Compose v2.20+ instalados de forma nativa.
- **Seguridad:** Certificado SSL/TLS válido (generado de forma automatizada mediante Let's Encrypt o Nginx Reverse Proxy).
- **Configuración:** Configuración del archivo `.env.production` conteniendo contraseñas complejas para base de datos y claves `JWT_SECRET` seguras.
- **Machine Learning:** Carga física del archivo `best_model.keras` dentro de la carpeta `/app/model/` del microservicio de ML.

5.4 Control de Versiones

El control de versiones de código y de imágenes del sistema se rige bajo la metodología Git Flow:

- **Rama principal (main):** Aloja únicamente código estable y liberado a producción. Cada despliegue se marca con una etiqueta de versión (v1.0.0, v1.0.1, etc.).
- **Rama de desarrollo (develop):** Rama de integración de cambios. Contiene los últimos desarrollos que han pasado pruebas de QA pero no se han liberado a producción.

- **Ramas de Hotfix (hotfix/*):**Ramas temporales creadas para reparar errores críticos de producción directamente y aplicarlos en caliente sobre la rama main.

5.5 Estrategia de Despliegue

Se implementará una estrategia de Despliegue Progresivo (Rolling Update) gestionada a través de Docker Compose. Durante la actualización de servicios (ej. el backend), se levantará un contenedor con la nueva versión y Nginx direccionará el tráfico una vez que el contenedor pase el healthcheck (pg_isready y endpoints de salud), apagando posteriormente la versión antigua. Esto minimiza el tiempo de inactividad a cero. Alternativamente, para actualizaciones mayores, se puede utilizar un esquema Blue-Green en servidores paralelos de staging y producción.

5.6 Validaciones Previas

Antes de ejecutar la orden de liberación en el servidor de producción, se deben realizar las siguientes pruebas de consistencia estática:

- **Análisis Estático:**Ejecutar npm run lint en frontend y backend para evitar errores de sintaxis o variables sin declarar.
- **Compilación Híbrida:**Validar la compilación limpia de la app móvil mediante Capacitor y la construcción limpia del directorio web (www).
- **Prueba de Inferencia:**Verificar que el modelo de machine learning sea importado correctamente mediante tensorflow y keras ejecutando un test unitario preliminar en FastAPI.

5.7 Lista de Verificación (Checklist) para Liberación

El Ingeniero DevOps a cargo de la liberación debe marcar obligatoriamente cada casilla antes de habilitar el tráfico de usuarios:

- **[] Seguridad de Puertos:**Verificar que el puerto 443 (HTTPS) esté abierto en el Firewall del VPS.
- **[] Entorno:**Habilitar la directiva NODE_ENV=production en el contenedor de backend.
- **[] Contraseñas de DB:**Generar contraseñas de alta seguridad (>16 caracteres) para PostgreSQL y reemplazar las claves por defecto.
- **[] Secreto JWT:**Asegurar que la clave JWT_SECRET del backend sea un hash complejo guardado en variables de entorno seguras.
- **[] Inicialización SQL:**Validar que el script inicial init.sql cree las tablas relacionales y el administrador inicial por defecto.
- **[] Montaje de Uploads:**Confirmar que la carpeta de almacenamiento de fotos compartidas esté montada sobre el volumen persistente Docker.
- **[] Rate Limit:**Confirmar que el rate limiter del backend esté configurado para mitigar ataques DDoS (100 req/15min).
- **[] Logs del Sistema:**Configurar la rotación de logs de Winston para evitar que se sature el almacenamiento de disco.

5.8 Gestión de Respaldos (Backups)

Para salvaguardar la información recolectada de brotes de plagas e información de agricultores, se implementará una política de respaldos automatizados de base de datos:

- **Automatización:** Un cron job programado en el servidor ejecutará diariamente a las 02:00 AM el comando `pg_dump` dentro del contenedor de base de datos `plagacontrol-postgres`.
- **Almacenamiento Off-site:** El dump resultante será comprimido en formato `.sql.gz` y se subirá de forma encriptada a un bucket de almacenamiento externo seguro (ej. AWS S3 o GCP Cloud Storage).
- **Retención:** Se mantendrá un esquema de retención de 30 días para respaldos diarios, y de 12 meses para respaldos mensuales.

5.9 Plan de Recuperación ante Fallos (Rollback)

En caso de presentarse una anomalía severa (ej. error 500 generalizado o base de datos inaccesible) tras el despliegue productivo, el equipo de liberación aplicará el siguiente procedimiento de reversión inmediata (máximo 15 minutos):

- **Paso 1:** Detener los contenedores actuales con `docker-compose down`.
- **Paso 2:** Hacer checkout a la última etiqueta de Git funcional (ej. `git checkout tags/v0.9.5`).
- **Paso 3:** Si hubo cambios incompatibles en los esquemas de base de datos, restaurar el respaldo de base de datos más reciente ejecutando `psql -U plaga_user -d plagacontrol < backup_reciente.sql` dentro del contenedor Postgres.
- **Paso 4:** Levantar nuevamente el sistema con `docker-compose up -d --build` y validar la salud de los contenedores.

5.10 Riesgos Conocidos

Se identifican los siguientes riesgos de infraestructura y despliegue junto con sus medidas de mitigación:

- **Incompatibilidad de memoria al cargar el modelo de Keras:** Mitigación: Asegurar que el contenedor de ML posea asignada memoria RAM dedicada en el host (>2 GB) y que TensorFlow use la compilación adecuada para el set de instrucciones del CPU del VPS.
- **Agotamiento de espacio en disco por imágenes de reportes:** Mitigación: Monitoreo preventivo del volumen de almacenamiento de fotos compartidas. Configurar alertas de uso de disco en el servidor VPS al superar el 85%.
- **Expiración de certificados TLS de Nginx:** Mitigación: Habilitar certificados auto-renovables con Certbot en el host y mapear las rutas de los certificados TLS directamente al contenedor de Nginx en modo lectura.

5.11 Responsables

Los roles involucrados en el ciclo de liberación están compuestos por:

- **Release Manager / DevOps:** Encargado de compilar los entregables y estructurar los contenedores Docker en el servidor VPS.

- **QA Engineer:** Responsable de verificar el correcto comportamiento de la app móvil y validar los flujos tras el despliegue.
- **Fullstack Developers:** Desarrolladores a cargo de generar hotfixes de código urgentes ante fallos.
- **Product Owner:** Responsable de dar el visto bueno comercial para la publicación de la versión.

5.12 Cronograma General de Liberación

El cronograma estructurado para la liberación de la v1.0.0 contempla:

- **T-24 Horas:** Congelamiento del repositorio de desarrollo y pruebas QA en el entorno de Staging.
- **T-6 Horas:** Configuración del entorno de producción, bases de datos y variables de entorno (.env.production).
- **T-0 Horas:** Ejecución del despliegue Docker en horario de mínimo tráfico (11:00 PM - 12:00 AM) y comprobación de Healthchecks.
- **T+1 Hora:** Monitoreo de logs mediante Winston y comprobación de métricas de base de datos en caliente.

5.13 Criterios de Aceptación

Para dar por concluido el proceso de liberación, se deben cumplir los siguientes hitos:

- **Cero errores críticos:** El 100% de las pruebas funcionales obligatorias (TC-01 a TC-10) deben finalizar con estado Aprobado.
- **Rendimiento óptimo:** El tiempo de procesamiento promedio del scanner IA debe ser menor a 2 segundos por imagen.
- **Cifrado SSL:** Toda comunicación externa con la aplicación debe realizarse forzosamente bajo cifrado HTTPS con TLS 1.2 o superior.

5.14 Recomendaciones para Futuras Versiones

- **1. Modo Desconectado (Offline Sync):** Integrar una base de datos SQLite interna en la app híbrida mediante plugins nativos de Capacitor para almacenar reportes sin conexión e irlos sincronizando progresivamente al recuperar la cobertura.
- **2. Automatización CI/CD:** Configurar un flujo automatizado de CI/CD en GitHub Actions que construya las imágenes Docker y las suba a Docker Hub de forma automática al hacer merge sobre la rama main.
- **3. Monitoreo Avanzado:** Implementar Prometheus junto con un panel visual en Grafana para analizar la tasa de peticiones y tiempos de respuesta de la base de datos relacional y el microservicio de inferencia de IA en producción.

6. CONCLUSIONES

La realización y documentación de este informe de aseguramiento de calidad, contenerización y plan de liberación de software para PlagaControl permiten consolidar las siguientes conclusiones profesionales de ingeniería:

6.1 Calidad del Software

El software PlagaControl cuenta con una base estructural de alta calidad, alineada con las directrices de arquitectura modular y limpia. El backend separa adecuadamente sus responsabilidades dividiendo rutas, validaciones con express-validator, lógica de negocio a través de controladores y mapeo de tablas relacionales con Sequelize. El frontend, estructurado en Angular, encapsula adecuadamente sus llamadas HTTP en servicios independientes inyectados, garantizando modularidad y facilitando su mantenimiento.

6.2 Resultados Obtenidos

La ejecución satisfactoria del plan de pruebas sobre 10 escenarios críticos e integraciones complejas certifica el comportamiento esperado de la aplicación. Se validó con éxito el bloqueo por segundo factor (MFA TOTP), la subida y redimensionado de imágenes compartidas, el mapeo dinámico de coordenadas en Leaflet y el tiempo óptimo de respuesta (menor a un segundo) de las inferencias convolucionales del modelo de machine learning en FastAPI. Las fallas encontradas (como PLG-BUG-001) son de severidad media-baja y cuentan con un plan de corrección claro.

6.3 Estado del Proyecto

El Proyecto Integrador PlagaControl se encuentra en un estado ESTABLE y LISTO para su liberación productiva. La infraestructura basada en contenedores Docker y orquestada por Docker Compose se encuentra correctamente enlazada con redes virtuales privadas y volúmenes de persistencia. Cumpliendo con los checklist de seguridad, rotación de logs de auditoría y variables de entorno de producción redactadas en este plan, el despliegue en un entorno en la nube puede llevarse a cabo con un riesgo sumamente reducido de inactividad o pérdida de datos.